极客时间

LangGraph 工作流

工程思想：报告式通信

第 3 周 · 第 10 节

讲师：黄佳

极客时间黄佳老师出品

极客时间

从模式到实现

模式选好了
用什么把它跑起来?

第 9 节选了 Orchestrator + Supervisor → 第 10 节用 LangGraph 实现

极客时间黄佳老师出品

第 9 节 vs 第 10 节：代码关系图

极客时间

patterns/ 是理解模式的 demo，workflows/ 是 LangGraph 生产版 —— 设计思想一脉相承，代码独立实现

第 9 节产出：patterns/

```
patterns/
  router.py
    classify_intent(query) -> str
    route(query) -> str
    # CLI 可直接运行

  supervisor.py
    Supervisor.execute(task) -> dict
    # Worker + Reviewer 循环
    # CLI 可直接运行
```

定位：教学 Demo

- 独立可运行 (python patterns/router.py)
- 接口：接受字符串，返回字符串
- 帮你理解 Router/Supervisor 模式

不 import

X

第 10 节产出：workflows/

```
workflows/
  state.py    KBState (TypedDict)
  nodes.py    5 个节点函数
    collect_node(state) -> dict
    analyze_node(state) -> dict
    organize_node(state) -> dict
    review_node(state) -> dict
    save_node(state) -> dict
  graph.py    StateGraph + 条件边
```

定位：LangGraph 生产版

- 接口：(state: KBState) –> dict
- 支持审核循环、条件边、状态管理
- 可扩展（Planner/CostGuard 加进来）

import

import **model_client.py**

唯一共享依赖：chat() / chat_json() / accumulate_usage()

极客时间黄佳老师出品

三种编排方式对比

极客时间

维度	纯 Python	Agent SDK	LangGraph
状态管理	手动（全局变量）	手动（context）	自动（StateGraph）
循环/重试	while 容易失控	自己实现	条件边 + 内置
可视化	无	无	自动生成流程图
持久化	手动写文件	无	Checkpoint 内置
适合场景	简单线性流程	灵活自定义	有循环的工作流

极客时间

本节工程思想

极客时间

报告式通信
Agent 间返回结构化摘要
而非完整执行日志

Agent 世界的 API Contract

报告式通信 — 量化对比

极客时间

传原始日志

Collector 返回原始 HTML/JSON
每次 Handoff ≈ 10,000 tokens
4 次 Handoff = 40,000 tokens

问题 1：成本高（¥0.04/次）
问题 2：下游 Agent 被噪音淹没
问题 3：容易超上下文窗口

传结构化摘要

Collector 返回 JSON 报告
每次 Handoff ≈ 500 tokens
4 次 Handoff = 2,000 tokens

优点 1：成本低（¥0.002/次）
优点 2：下游 Agent 准确理解
优点 3：远离上下文限制

节省 20 倍 Token

报告式通信 — 代码体现

极客时间

projects/v3-multi-agent/workflows/state.py | python

```python
from typing import TypedDict

class KBState(TypedDict):
    """知识库工作流的全局状态 — 每个字段都是一份报告"""
    sources: list[dict]          # 原始采集数据（GitHub API / RSS）
    analyses: list[dict]         # LLM 分析后的结构化结果
    articles: list[dict]         # 格式化、去重后的知识条目
    review_feedback: str         # 审核 Agent 的反馈意见
    review_passed: bool          # 审核是否通过
    iteration: int               # 当前审核循环次数（最多 3 次）
    cost_tracker: dict           # Token 用量追踪

# 设计原则：
# 1. 每个节点只读自己需要的字段
# 2. 每个节点只写自己负责的字段
# 3. 字段值是摘要，不是原始数据
# → 这就是报告式通信的 LangGraph 实现
```

极客时间黄佳老师出品

LangGraph ≠ LangChain

- **常见误解：** LangGraph 是 LangChain 的一部分

- **事实：** LangGraph 是独立的包
- pip install langgraph # 只需要这一个依赖
- 不需要 langchain / langchain-core / langchain-community

- **我们的组合：**
- LangGraph — 管图编排（状态 + 节点 + 边 + 循环）
- model_client.py — 管模型调用（quick_chat() 函数，第 6 节已写好）
- 不用 langchain-anthropic / langchain-openai / ChatModel

- **为什么不用 LangChain 的 ChatModel?**
- model_client.py 已经封装了多模型切换和成本追踪
- 不想引入额外的抽象层和依赖
- KISS 原则：能不加的依赖就不加

极客时间黄佳老师出品

LangGraph 核心概念 — 四大组件

极客时间

StateGraph
（有状态的有向图）

State（共享状态）	Node（处理节点）	Edge（连接）	Checkpoint（状态持久化）
TypedDict 所有节点 共享读写	Python 函数 读取 state 返回更新	普通/条件 决定走向 支持循环	每步自动保存 决定走向 支持循环 断点恢复

Node — 采集节点 + 分析节点

极客时间

projects/v3-multi-agent/workflows/nodes.py | python

```python
from model_client import quick_chat

def collect_node(state: KBState) -> dict:
    """采集节点 — 返回结构化报告"""
    # 调用 GitHub API + RSS，返回结构化数据
    items = collect_github(limit=20) + collect_rss(limit=20)
    return {
        "sources": items
    }

def analyze_node(state: KBState) -> dict:
    """分析节点 — 基于采集报告做分析"""
    results = []
    for item in state["sources"]:
        prompt = f"分析以下技术内容：\n{item}\n"\
                 f"输出 JSON：摘要、亮点、评分(1-10)、标签。"
        analysis = quick_chat(prompt, provider="deepseek")
        results.append(analysis)
    return {"analyses": results}
```

Edge — 条件边与审核路由

projects/v3-multi-agent/workflows/graph.py | python

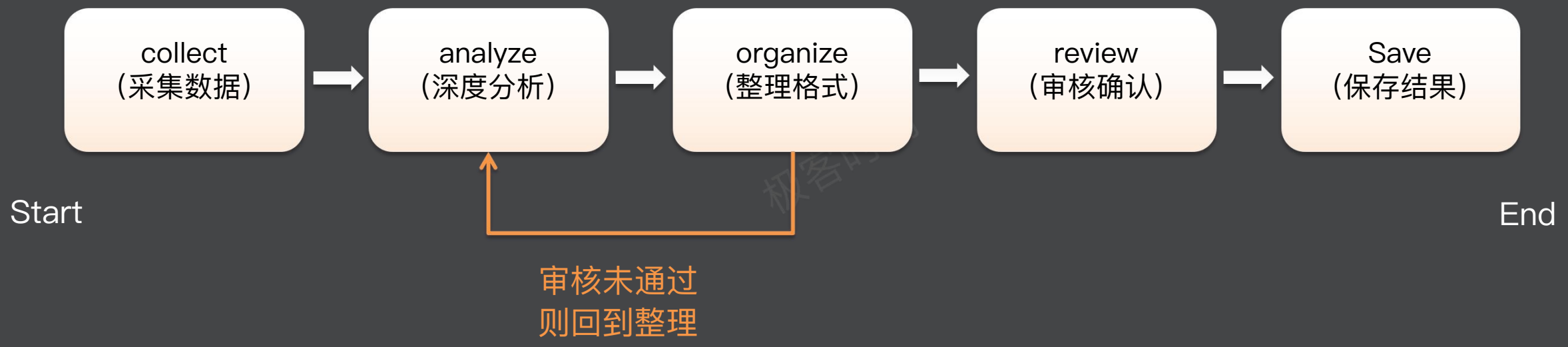
```python
from langgraph.graph import StateGraph, END

def should_continue(state: KBState) -> str:
    """条件路由 — 审核后决定下一步"""
    if state.get("review_passed", False):
        return "save"          # 审核通过
    else:
        return "organize"      # 未通过，回到整理

# 注意：最大迭代次数（3次）的强制通过
# 逻辑在 review_node 内处理

# 条件边：审核后的两条路
graph.add_conditional_edges(
    "review",                  # 从哪出发
    should_continue,           # 路由函数
    {
        "save": "save",        # 通过 → 保存
        "organize": "organize", # 未通过 → 重新整理
    }
)
```

知识库 V3 完整工作流



极客时间黄佳老师出品

审核节点 — review_node

极客时间

projects/v3-multi-agent/workflows/nodes.py | python

```python
import json

def review_node(state: KBState) -> dict:
    """审核节点 — 评估文章质量，返回通过/不通过+反馈"""
    articles = state["articles"]
    iteration = state.get("iteration", 0) + 1
    prompt = f"""评估以下知识条目质量：
{articles}

评估维度：准确性、分析深度、结构清晰度、可操作性
输出 JSON：{{\"passed\": true/false, \"feedback\": \"改进建议\"}}
只输出 JSON。"""
    reply = quick_chat(prompt, provider="deepseek")
    try:
        result = json.loads(reply)
        passed = result["passed"] or iteration >= 3
        return {"review_passed": passed,
                "review_feedback": result["feedback"],
                "iteration": iteration}
    except json.JSONDecodeError:
        return {"review_passed": iteration >= 3,
                "review_feedback": "解析失败，建议修改",
                "iteration": iteration}
```

整理节点 + 保存节点

极客时间

projects/v3-multi-agent/workflows/nodes.py | python

```python
def organize_node(state: KBState) -> dict:
    """整理节点 — 格式化、去重，生成知识条目"""
    feedback = state.get("review_feedback", "")
    analyses = state["analyses"]
    prompt = f"将以下分析结果整理为知识条目：\n{analyses}\n"
    if feedback:
        prompt += f"审核反馈（请据此改进）：{feedback}\n"
    prompt += "输出 JSON 数组，每条含 title/summary/tags/score。"
    result = quick_chat(prompt, provider="deepseek")
    return {
        "articles": json.loads(result) if result else []
    }

def save_node(state: KBState) -> dict:
    """保存节点 — 将知识条目写入文件"""
    articles = state["articles"]
    for article in articles:
        # 写入 data/ 目录
        filename = f"data/{article['title']}.json"
        with open(filename, "w") as f:
            json.dump(article, f, ensure_ascii=False)
    return {"cost_tracker": state.get("cost_tracker", {})}
```

Graph 组装 — 30 行搞定

极客时间

projects/v3-multi-agent/workflows/graph.py | python

```python
from langgraph.graph import StateGraph, END

def build_graph():
    graph = StateGraph(KBState)

    # 添加 5 个节点
    graph.add_node("collect", collect_node)
    graph.add_node("analyze", analyze_node)
    graph.add_node("organize", organize_node)
    graph.add_node("review", review_node)
    graph.add_node("save", save_node)

    # 线性边
    graph.add_edge("collect", "analyze")
    graph.add_edge("analyze", "organize")
    graph.add_edge("organize", "review")
    graph.add_edge("save", END)
```

projects/v3-multi-agent/workflows/graph.py | python

```python
    # 条件边：审核路由
    graph.add_conditional_edges(
        "review", should_continue,
        {"save": "save",
         "organize": "organize"})  # 循环！

    graph.set_entry_point("collect")
    return graph.compile()
```

运行工作流 — invoke

projects/v3-multi-agent/workflows/graph.py | python

```python
app = build_graph().compile()

# 初始状态
initial_state: KBState = {
    "sources": [],
    "analyses": [],
    "articles": [],
    "review_feedback": "",
    "review_passed": False,
    "iteration": 0,
    "cost_tracker": {},
}
```

projects/v3-multi-agent/workflows/graph.py | python

```python
# 流式执行，观察每个节点的输出
for event in app.stream(initial_state):
    node_name = list(event.keys())[0]
    print(f"--- [{node_name}] 完成 ---")
    node_output = event[node_name]
    if "sources" in node_output:
        print(f"  采集数量:
{len(node_output['sources'])}")
    if "review_passed" in node_output:
        passed = node_output['review_passed']
        print(f"  审核: {'通过' if passed else '未通过'}")
```

极客时间黄佳老师出品

工程思想总结卡 Checkpoint 三大能力

- **1. 断点恢复**
- 跑到 Step 3 出错？修复后从 Step 3 继续
- 不用重跑 Step 1–2（省时间、省 Token）
- **2. 状态审计**
- 每一步的 state 都有记录
- 出了问题可以回溯：是采集数据不好？还是分析有误？
- **3. 并发隔离**
- 不同 thread_id = 不同工作流实例
- 用户 A 和用户 B 同时跑，状态互不干扰
- **开发 vs 生产：**
- 开发：MemorySaver（内存，重启丢失）
- 生产：SqliteSaver / PostgresSaver（持久化）

极客时间黄佳老师出品

Human-in-the-Loop — 渐进信任

- **interrupt_before — 在关键节点暂停等待人工确认**
- app = graph.compile(interrupt_before=["save"])
- 工作流跑到 save 前自动暂停
- 人工检查无误后 → app.invoke(None, config) 继续

- **渐进信任策略：**

  Week 1（不信任）：
- interrupt_before = ["analyze", "review", "save"]
- 每步都让人确认

  Week 2（建立信任）：
- interrupt_before = ["save"]
- 只在最终输出前确认

  Week 3（充分信任）：
- interrupt_before = []
- 全自动运行

极客时间黄佳老师出品

本节知识图谱

极客时间

概念	作用	知识库应用
StateGraph	有状态的图	KBState 定义 7 个字段 (sources/analyses/articles...)
Node	处理节点	5 个 Agent 节点
Edge	节点连接	4 条普通边
Conditional Edge	动态路由	审核通过/失败/超限
Checkpoint	状态持久化	断点恢复 + 并发隔离
interrupt_before	人工确认	渐进信任策略

本节作业

极客时间

任务 1：定义 KBState 状态

安装 LangGraph，创建 workflow/state.py。定义 KBState TypedDict，包含 7 个字段。
验证：Python 能正常 import KBState 且类型检查通过。

任务 2：搭建 5 节点工作流

实现 collect/analyze/organize/review/save 五个节点。组装 StateGraph，添加边，compile 后 invoke 运行。
验证：运行后 articles 列表非空且数据已保存。

任务 3：添加审核循环

添加 should_continue 条件边。测试三种场景：首次通过、修改 1 次通过、修改 3 次强制通过。
验证：iteration 和预期一致。

极客时间

本节小结

LangGraph = 有状态的图引擎，天生处理循环

报告式通信 = 传结构化摘要，节省 20 倍 token

5 节点 + 条件边 + 审核循环 = V3 核心架构

下一节 第 11 节：多 Agent 数据流 — 从单工作流到多源协作

极客时间黄佳老师出品